

Reporte de implementación.

Mauricio Olguín Sánchez - A01711522

29 de agosto de 2026

Índice general

1. Realizando la implementación «A manita»

- 1.1. Sobre la base de datos seleccionada
- 1.2. Proceso de ETL
 - 1.2.1. *Extraction*
 - 1.2.2. *Transforming*
 - 1.2.3. *Loading*
- 1.3. *Exploratory Data Analysis*
 - 1.3.1. Conclusiones
- 1.4. Modelo de regresión lineal sin frameworks
 - 1.4.1. Normalización de los datos
 - 1.4.2. Implementación del modelo
 - 1.4.3. Uso de los conjuntos de entrenamiento, validación y prueba
 - 1.4.4. Resultados de este primer modelo

2. Anexo

- 2.1. Figuras amplias
- 2.2. Códigos

Bibliografía

Capítulo 1

Realizando la implementación «Amanita»

1.1. Sobre la base de datos seleccionada

Después de haber investigado en páginas como [UC Irvine](#), [Kaggle](#), el [World Bank Open Data](#), entre otros, encontré en la página de [OpenML](#) un *dataset* interesante. Tras aplicar varios filtros, encontré la base de datos [Airlines_DepDelay_10M](#).

Esta base de datos está disponible en OpenML y está construida a partir de datos provenientes de la Oficina de Estadísticas de Transporte de los Estados Unidos (*Bureau of Transportation Statistics*, BTS), específicamente de la base de datos *Airline On-Time Performance Data*.

La base de datos seleccionada contiene 10,000,000 de registros y 10 variables, y corresponde a un problema de regresión. A continuación, presento una breve descripción general del *dataset*:

Propósito: Predecir el tiempo de retraso en la salida de los vuelos, medido en minutos (*Departure Delay*).

Fecha de los datos: Desde 1987 hasta 2013.

Formato: ARFF (*Attribute-Relation File Format*).

Tipo de tarea: Regresión.

Número de registros: 10,000,000.

Número de variables: 10.

Aunque OpenML proporciona información sobre cada variable, dentro de la fuente original (en el [Bureau of Transportation Statistics](#)), existe una descripción más detallada sobre los datos:

DepDelay - numerico. Diferencia, en minutos, entre la hora de salida programada y la hora de salida real del vuelo. Los valores negativos indican que el vuelo salió antes de lo programado.

Month - numerico. Mes en el que se realizó el vuelo, con valores del 1 al 12, donde 1 corresponde a enero y 12 a diciembre.

DayofMonth - numerico. Día del mes en el que se realizó el vuelo, con valores del 1 al 31.

DayOfWeek - numerico. Día de la semana en el que se realizó el vuelo, representado mediante valores del 1 al 7.

CRSDepTime - numerico. Hora de salida programada del vuelo según el sistema de reservas computarizado de la aerolínea (CRS), expresada en hora local con formato HHMM.

CRSArrTime - numerico. Hora de llegada programada del vuelo según el sistema de reservas computarizado de la aerolínea (CRS), expresada en hora local con formato HHMM.

UniqueCarrier - categorico. Código único utilizado para identificar a la aerolínea que opera el vuelo.

Origin - categorico. Código del aeropuerto de origen del vuelo.

Dest - categorico. Código del aeropuerto de destino del vuelo.

Distance - numerico. Distancia total del vuelo, expresada en millas.

En contraste, las variables *UniqueCarrier*, *Origin* y *Dest* son explícitamente categóricas, pero serán excluidas del modelo. Su transformación (utilizando la misma técnica) produciría un incremento considerablemente mayor en la cantidad de variables. Principalmente, tomo esta decisión con el objetivo de mantener un tiempo de ejecución razonable y centrar el proyecto en la implementación manual del algoritmo de regresión.

1.2. Proceso de ETL

1.2.1. *Extraction*

Como ya lo he mencionado antes, los datos se han obtenido del siguiente link: <https://www.openml.org/search?type=data&status=active&id=42728>

Sin embargo, si se quisiera obtener más datos actuales, habría que descargarlos (sea de forma manual o creando algún *script*) desde el siguiente link: <https://transtats.bts.gov/PREZIP/>.

Aquí, se recopilan todos los datos desde 1987 hasta junio de 2026.

1.2.2. *Transforming*

Las variables *Month*, *DayofMonth* y *DayOfWeek*, aunque se encuentran representadas numéricamente en el conjunto de datos, las trataré como variables categóricas. Esto se debe a que, en realidad, los valores numéricos representan categorías y no implican necesariamente una relación lineal entre ellas. Por ejemplo, el mes representado por el valor 12 no debe interpretarse como una categoría doce veces mayor que el mes representado por el valor 1.

Para representar estas variables dentro del modelo de regresión, utilizaré la técnica de *One-Hot Encoding*. Aunque esto implicará que crezca la dimensionalidad del conjunto de datos.

Sobre los atributos *CRSDepTime* y *CRSArrTime*

Las variables *CRSDepTime* y *CRSArrTime* representan las horas programadas de salida y llegada de los vuelos. En el conjunto de datos, estas se representan en el formato HHMM, es decir, que si la hora era 13:20, en los datos se muestran 1320.

Sin embargo, esta representación no resulta adecuada al momento de utilizarla directamente como valores numéricos porque, dentro del modelo de regresión, no será representado la distancia real. Por ejemplo, los valores de 2355 y 5, aunque están apartados por 10 minutos, utilizando los datos tal cual como vienen en el conjunto de datos, estos estarían muy separados entre sí.

Es por esto que ambas variables tienen que ser transformadas y, la mejor conversión que he podido investigar es utilizar como referencia la medianoche. Para realizar esta conversión, se separó la hora y los minutos del valor original y se aplicó la siguiente expresión:

$$\text{Minutos}_{\text{desde la media noche}} = 60 \times \text{Hora} + \text{Minutos}$$

De esta forma, conservamos la información temporal original y proporcionamos una representación numérica más adecuada para el modelo de regresión.

Sobre el formato del archivo de los datos

Al empezar a realizar una exploración de los datos, cada que tenía que ejecutar el programa para visualizarlos, tardaba un aproximado de 45 segundos cada que ejecutaba el programa. Investigando opciones para solucionar este problema, me encontré con el formato parquet. Este formato está precesamente optimizado para mejorar el manejo de datos complejos, el rendimiento de las consultas que se hacen y la eficiencia del almacenamiento [1].

Esta integración, al ver que me resultaría óptima para poder realizar más pruebas más rápidas, la he implementado. Así pues, el archivo ha pasado de tener un peso de 385 MB a sólo 56 MB; además de que el tiempo de carga se redujo drásticamente de 45 segundos a 2 segundos o menos.

1.2.3. Loading

Una vez hechas las transformaciones anteriores, ahora dividiré el tamaño de mi *dataset* en tres principales secciones: *train*, *validation* y *test*. Al ser la cantidad de datos tan alta, podría realizar varias particiones para *validation*, sin embargo, en esta primera parte en donde me enfoco aprender el algoritmo, sólo haré una vez esta división. Así pues, utilizaré una proporción de 98 % de los datos para *train*, 1 % para *validation* y 1 % para *test*. [2]

1.3. Exploratory Data Analysis

Hice un primer acercamiento general a los datos simplemente realizando un gráfico de pares (*pairplot*), el cuál resultó en el siguiente: (ver figura 1.1)

De estos gráficos, se puede observar que, cuando se comparan los datos entre *DepDelay*, se pueden apreciar una posible zona donde la mayoría de los datos recaen, sin embargo, también es interesante ver que

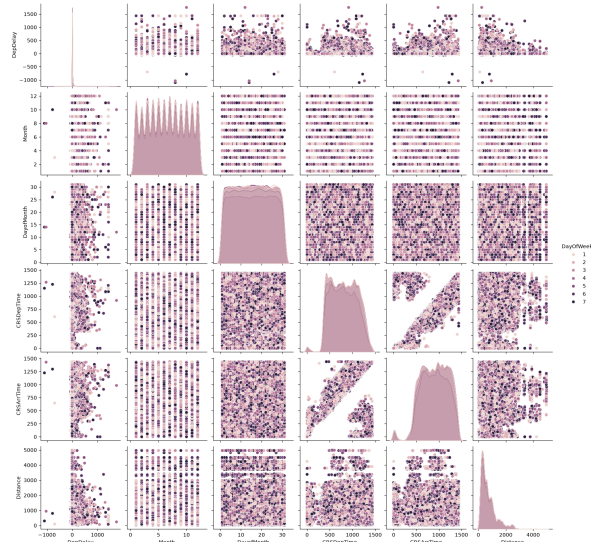


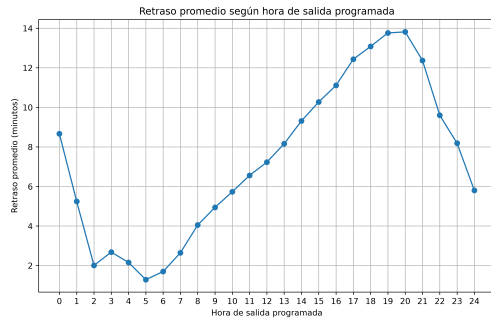
Figura 1.1: *Pair plot* para visualizar la mayoría de los datos comparados entre sí. No se graficaron los 10 millones de puntos por el tiempo de computo requerido, sólo se graficaron un millón. Se añadió también que, para cada punto, se etiquetara por su el día de la semana (*DayOfWeek*) para intentar de visualizar alguna zona de interes así como la distribución de los mismos.

existen outliers negativos, es decir, que hubo días en donde un vuelo se adelantó a la hora de su salida. También, podemos observar que entre las variables de *CRSDepartTime* y *CRSArrTime* existe una clara agrupación de los datos, por lo que valdría la pena volver a realizar este gráfico pero ahora también comparandolo con la distancia final y/o con el tiempo de retraso.

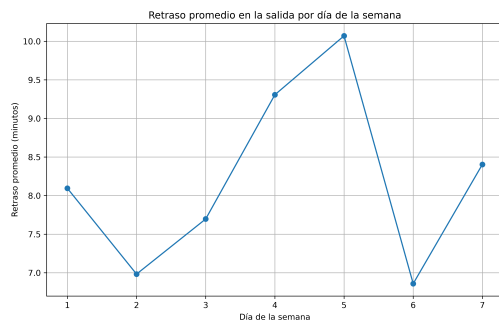
Ahora que tenemos el panorama general de los datos, podría pasar a realizar gráficas específicas que me cuenten un poco más sobre los mismos datos. Al contar con los datos de los días de la semana, días del mes, meses del año y las horas en las que se retrasan, decidí hacer un gráfico con los minutos promedio por cada serie de tiempo para ver cómo es que evolucionan los retrasos durante cada periodo de tiempo (ver figura 1.2)

De cada uno, puedo destacar lo siguiente:

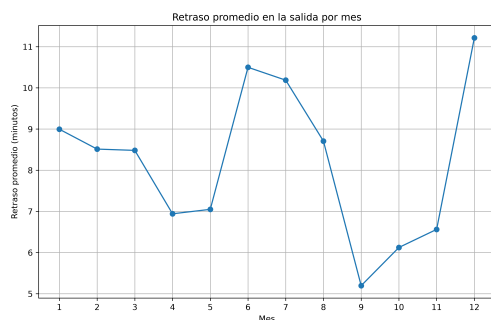
- Para los retrasos por cada hora del día se puede apreciar que, si empezamos a ver la gráfica desde las 5 de la mañana (el punto más bajo en promedio de retrasos), por cada hora que pasa va in-



(a) Promedios de retrasos por cada hora.



(b) Promedios de retrasos por cada día de la semana.



(c) Promedios de retrasos por cada mes.

Figura 1.2: Promedios de retrasos por periodos de tiempos.

criementando el retraso en cada vuelo hasta llegar al punto máximo hasta las 7 de la tarde (19 horas).

- Algo similar a lo anterior pasa a lo largo de la semana pues, a medida que se acerca el fin de semana, los vuelos cada día se retrasan más, hasta que el viernes llega a su punto máximo.
- Por último, durante el verano e inicios de años es cuando existen más retrasos en los vuelos. Aunque esta información me hace pensar que coincide con perio-

dos vacacionales, no puedo afirmar con la información que tengo que esta sea la razón.

Sin embargo, es importante destacar que estos valores podrían estar altamente afectados por los *outliers* que existen para cada serie de tiempo. Para confirmar esto, vale entonces la pena realizar el histograma específico con los retrasos (ver figura 1.3).



Figura 1.3: Histograma con retrasos en los vuelos. **NOTA:** La escala de este histograma se ha recortado de -25 a 150. Esto debido a que los *outliers* (tanto positivos como negativos) son tan grandes y muy pocos que reducen mucho la visibilidad de la gran mayoría de los datos.

Es interesante observar que la inmensa mayoría de vuelos está a tiempo, ya que es en donde más se concentran los datos. Sin embargo, también es interesante ver que existen vuelos que se adelantan.

Por último, el gráfico más interesante en donde podemos comparar y darnos cuenta de la relación lineal entre las variables es una matriz de correlación (ver figura 1.4). Como se puede observar en la matriz, existe una correlación lineal débil entre la mayoría de las variables independientes y la variable objetivo *DepDelay*. Las excepciones relativas son *CRSDepTime* y *CRSArrTime*, que presentan los valores positivos más altos (0.12 y 0.11 respectivamente), aunque su poder predictivo lineal sigue siendo moderado[cite: 3]. Asimismo, se detectó una fuerte multicolinealidad (0.70) entre *CRSDepTime* y *CRSArrTime*; un comportamiento estadísticamente esperado dado que repre-

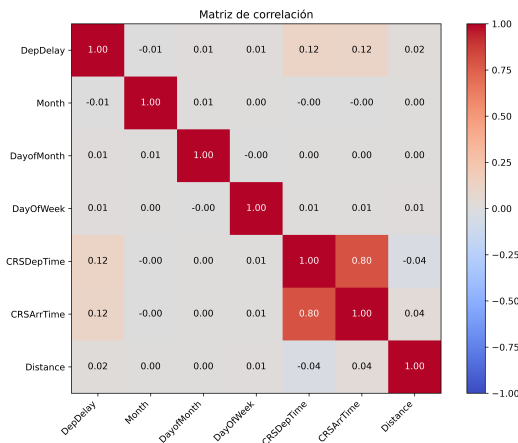


Figura 1.4: Matriz de correlación.

sentan ventanas temporales lógicamente secuenciales de un mismo vuelo.

1.3.1. Conclusiones

A partir de este análisis, podemos determinar que:

Naturaleza de los datos: La baja correlación generalizada sugiere que la relación que dicta los retrasos no es estrictamente lineal. Esto nos permite justificar desde ahora si el modelo de Regresión Lineal obtiene un coeficiente de determinación (R^2) moderado, ya que el algoritmo hará su mejor esfuerzo sobre una base de datos compleja.

Multicolinealidad temporal: La fuerte relación entre las horas de salida y llegada implica que ambas variables aportan información redundante al modelo, lo cual podría generar cierta inestabilidad al momento de que el algoritmo asigne los pesos.

Disparidad de magnitudes: Las variables independientes operan en escalas drásticamente distintas, contrastando vuelos de miles de millas físicas contra el formato temporal de horas y minutos.

Por estas razones, y para resolver específicamente la diferencia de magnitudes, es indispensable proceder con la normalización

de los datos. Este paso asegurará que todas las características numéricas convivan en un rango equitativo. Al nivelar la escala, evitamos que las variables con valores más grandes dominen el cálculo de la función de costo, permitiendo que el algoritmo del *Descenso de Gradiente* converja hacia los pesos óptimos de manera mucho más rápida y estable. En la siguiente sección, hablaré un poco más a detalle sobre los algoritmos que aplicaré sobre mi modelo y su función dentro del mismo.

1.4. Modelo de regresión lineal sin frameworks

1.4.1. Normalización de los datos

El algoritmo a utilizar para la normalización de los datos será *Min-Max*:

$$X_{norm} = \frac{X - X_{min}}{X_{max} - X_{min}}$$

Gracias a este algoritmo, lograremos poner en una escala del 0 a 1 todos nuestros datos y, como ya se ha dicho anteriormente, durante la aplicación del *Descenso del Gradiente* converja más rápido.

Hay que tener cuidado a qué variables les aplicamos la normalización de los datos pues no es necesario hacerlo para todas las variables. Además, la normalización se debe de hacer con respecto a los datos de entrenamiento, pero aplicárselo a todos los conjuntos de datos (*test* y *validation*). Como anteriormente he aplicado un *One Hot Encoding* para las variables temporales, entonces, las únicas columnas a las que se les aplicará la normalización serán a *CRSDepTime*, *CRSArrTime* y *Distance*.

1.4.2. Implementación del modelo

Una vez preparados y normalizados los datos, ahora podemos pasar con la implementación del modelo de regresión. El modelo

utilizado parte de una combinación lineal de las variables predictoras. Para una observación con n características, la predicción se expresa como:

$$\hat{y} = b + \sum_{j=1}^n w_j x_j \quad (1.1)$$

donde $\hat{y}$ representa la predicción del modelo, b corresponde al *bias* o intercepto, w_j representa el peso asociado a la característica x_j y n corresponde al número total de variables predictoras.

Función de hipótesis

La primera función implementada corresponde a la hipótesis del modelo. Su función es recibir las características de una observación, los pesos actuales y el *bias*, para obtener una predicción.

$$h(x) = b + \sum_{j=1}^n w_j x_j \quad (1.2)$$

Esta función constituye la base del resto de la implementación, ya que tanto el cálculo del error como el cálculo de los gradientes dependen de las predicciones generadas por la hipótesis.

Cálculo del error

Una vez obtenida una predicción, se calcula la diferencia entre el valor predicho y el valor real:

$$e_i = \hat{y}_i - y_i \quad (1.3)$$

El signo del error es importante durante el entrenamiento, ya que posteriormente será utilizado para determinar la dirección en la que deben modificarse los parámetros del modelo.

Función de costo

Para medir qué tan alejadas se encuentran las predicciones respecto a los valores reales se utilizó el error cuadrático medio (*Mean Squared Error*, MSE).

La función de costo utilizada es:

$$J(w, b) = \frac{1}{m} \sum_{i=1}^m (\hat{y}_i - y_i)^2 \quad (1.4)$$

donde m representa el número de observaciones utilizadas para el entrenamiento.

Elevar los errores al cuadrado permite evitar que los errores positivos y negativos se cancelen entre sí y penaliza en mayor medida los errores de mayor magnitud. El objetivo del *Descenso de Gradiente* será entonces encontrar los valores de w y b que minimicen esta función.

Gradiente respecto a los pesos

Para poder modificar los parámetros del modelo es necesario determinar cómo cambia la función de costo cuando se modifica cada uno de los pesos. La derivada parcial del costo respecto a un peso w_j es:

$$\frac{\partial J}{\partial w_j} = \frac{2}{m} \sum_{i=1}^m (\hat{y}_i - y_i) x_{ij} \quad (1.5)$$

Durante la implementación se recorre cada peso y, para cada uno de ellos, se recorren todas las observaciones del conjunto de entrenamiento. Para cada observación se calcula su predicción, se obtiene su error y posteriormente se multiplica dicho error por el valor correspondiente de la característica. El resultado acumulado representa la dirección y magnitud en la que debe modificarse cada peso para disminuir la función de costo.

Gradiente respecto al *bias*

El *bias* también necesita actualizarse durante el entrenamiento. La derivada de la función de costo respecto al *bias* es:

$$\frac{\partial J}{\partial b} = \frac{2}{m} \sum_{i=1}^m (\hat{y}_i - y_i) \quad (1.6)$$

A diferencia del gradiente de los pesos, en este caso no se multiplica el error por ninguna característica. Esto se debe a que la

derivada de la hipótesis respecto al *bias* es igual a 1.

Actualización de los parámetros

Una vez calculados los gradientes, los parámetros del modelo son actualizados mediante el algoritmo de Descenso de Gradiente. La actualización de cada peso se realiza mediante:

$$w_j^{nuevo} = w_j - \alpha \frac{\partial J}{\partial w_j} \quad (1.7)$$

Mientras que el *bias* se actualiza mediante:

$$b^{nuevo} = b - \alpha \frac{\partial J}{\partial b} \quad (1.8)$$

donde α representa la tasa de aprendizaje (*learning rate*).

El signo negativo es fundamental, ya que el gradiente indica la dirección de mayor incremento de la función de costo. Por lo tanto, se avanza en la dirección contraria para buscar un mínimo de la función.

Proceso de entrenamiento

Las funciones anteriores se integraron en un ciclo de entrenamiento. En cada época se realizan las siguientes operaciones:

1. Generar las predicciones utilizando los parámetros actuales.
2. Calcular la función de costo.
3. Obtener el gradiente de cada peso.
4. Obtener el gradiente del *bias*.
5. Actualizar todos los pesos.
6. Actualizar el *bias*.
7. Guardar el costo obtenido para analizar la evolución del modelo.

1.4.3. Uso de los conjuntos de entrenamiento, validación y prueba

Para evaluar el comportamiento del modelo se utilizaron tres conjuntos de datos: *train*, *validation* y *test*.

El conjunto de *train* es el único utilizado para modificar los pesos y el *bias*. En cada época, el algoritmo calcula los gradientes a partir de este conjunto y actualiza los parámetros.

El conjunto de *validation* no participa en la actualización de los parámetros. Después de cada época, se utilizan los parámetros obtenidos hasta ese momento para generar predicciones sobre el conjunto de validación y calcular su correspondiente función de costo. Esto permite observar cómo generaliza el modelo sobre datos que no fueron utilizados directamente para actualizar sus parámetros.

Finalmente, el conjunto de *test* se mantiene separado durante el proceso de entrenamiento. Una vez finalizado el entrenamiento, se utilizan los parámetros finales del modelo para generar predicciones sobre este conjunto y obtener una evaluación final del desempeño.

Esta separación permite distinguir entre el desempeño sobre los datos utilizados para aprender y el desempeño sobre datos que el modelo no utilizó para ajustar sus parámetros.

Monitoreo del aprendizaje y detección de *overfitting*

Para observar el comportamiento del modelo durante el entrenamiento se almacena la función de costo correspondiente a cada época. Posteriormente, se pueden representar las curvas de costo de *train* y *validation* en una misma gráfica.

La finalidad principal de esta comparación es detectar un posible *overfitting*. Si el costo de entrenamiento continúa disminuyendo mientras que el costo de validación comien-

za a incrementarse, esto indicaría que el modelo está ajustándose progresivamente a las características particulares de los datos de entrenamiento y perdiendo capacidad de generalización.

Por el contrario, si ambos costos disminuyen de manera similar y permanecen relativamente cercanos, se tendría evidencia de un comportamiento de generalización más estable.

El conjunto de *test* no se utiliza para este monitoreo durante las épocas, sino únicamente para realizar la evaluación final del modelo.

1.4.4. Resultados de este primer modelo

Implementación de Mini-Batch

Debido al tamaño del conjunto de datos, se decidió utilizar el *Descenso de Gradiente por Mini-Batch*. El conjunto de entrenamiento contiene aproximadamente 9.8 millones de observaciones, por lo que calcular los gradientes utilizando todos los datos antes de cada actualización requeriría un tiempo de cómputo considerable.

El uso de mini-lotes permite dividir este proceso en grupos pequeños y realizar actualizaciones más frecuentes de los parámetros. De esta manera, se reduce la cantidad de datos procesados en cada actualización y se mejora el tiempo de entrenamiento sin utilizar únicamente una observación a la vez, lo que proporciona un proceso de aprendizaje más estable que el *Stochastic Gradient Descent*.

Para esta implementación se utilizó un tamaño de lote de 256 observaciones. En cada época, el conjunto de entrenamiento se divide en estos lotes y, después de procesar cada uno, se actualizan los pesos y el *bias*. Al finalizar la época, se calcula el costo promedio de los lotes procesados para monitorear la evolución del modelo.

La principal razón para utilizar esta técnica fue, por tanto, hacer viable el entrenamiento del modelo sobre un conjunto de datos de 10 millones de registros, reduciendo el costo computacional de cada actualización.

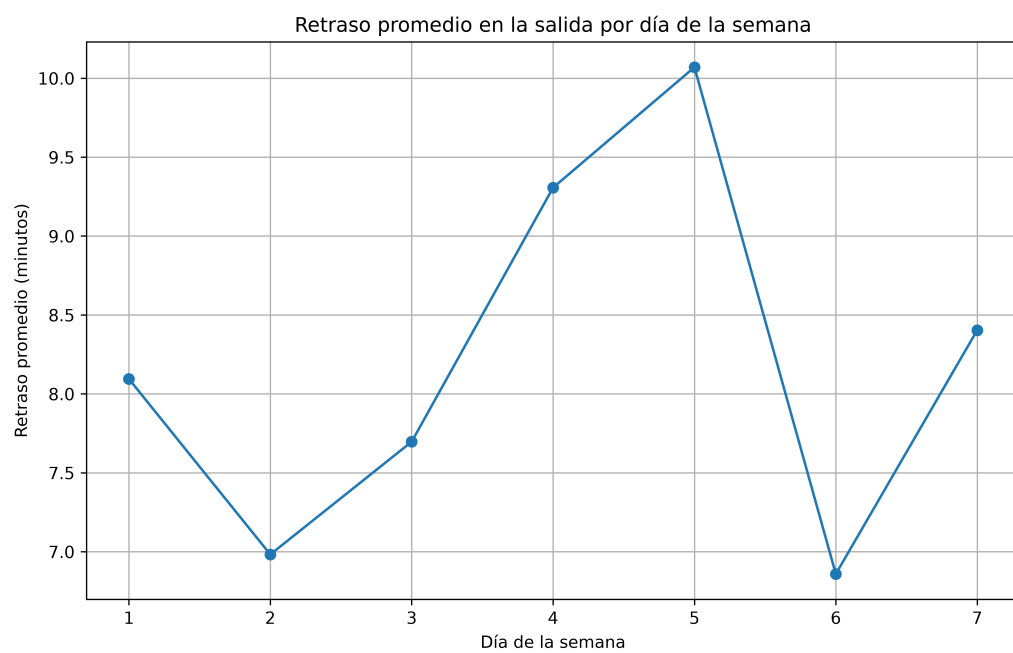
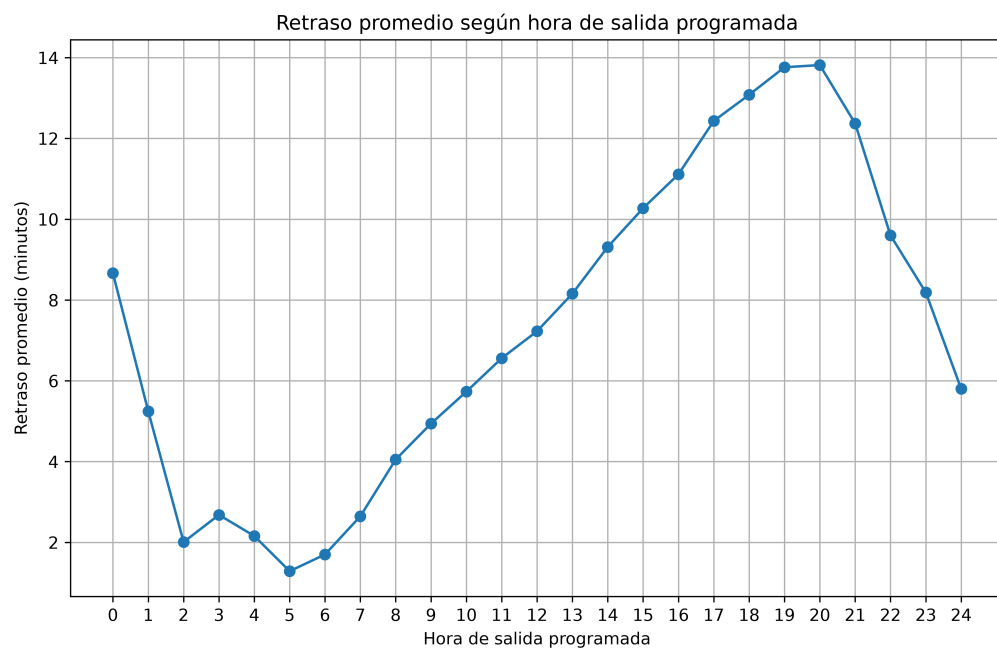
Capítulo 2

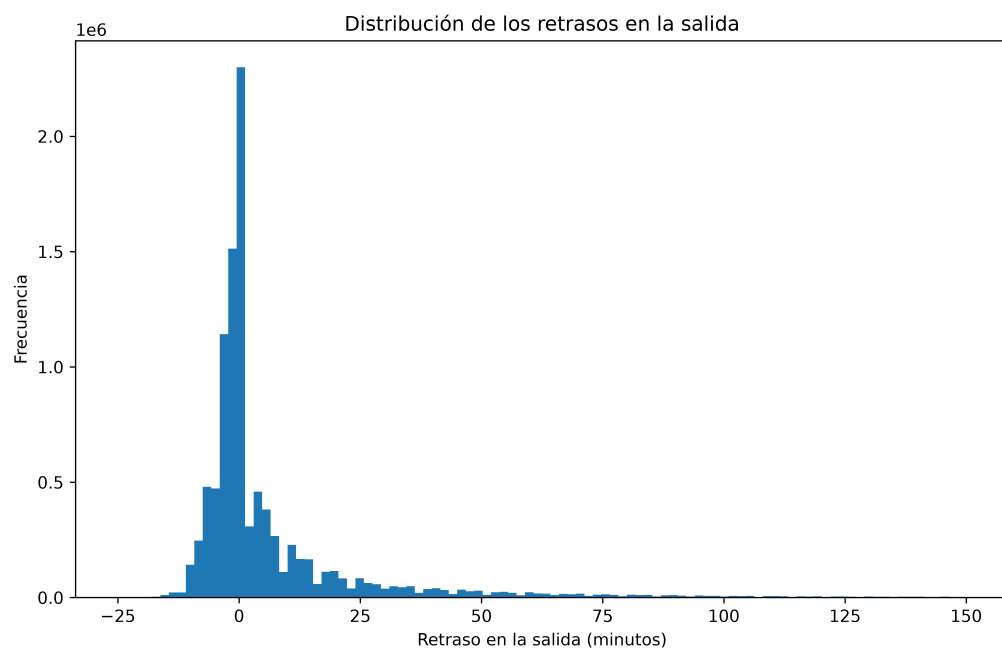
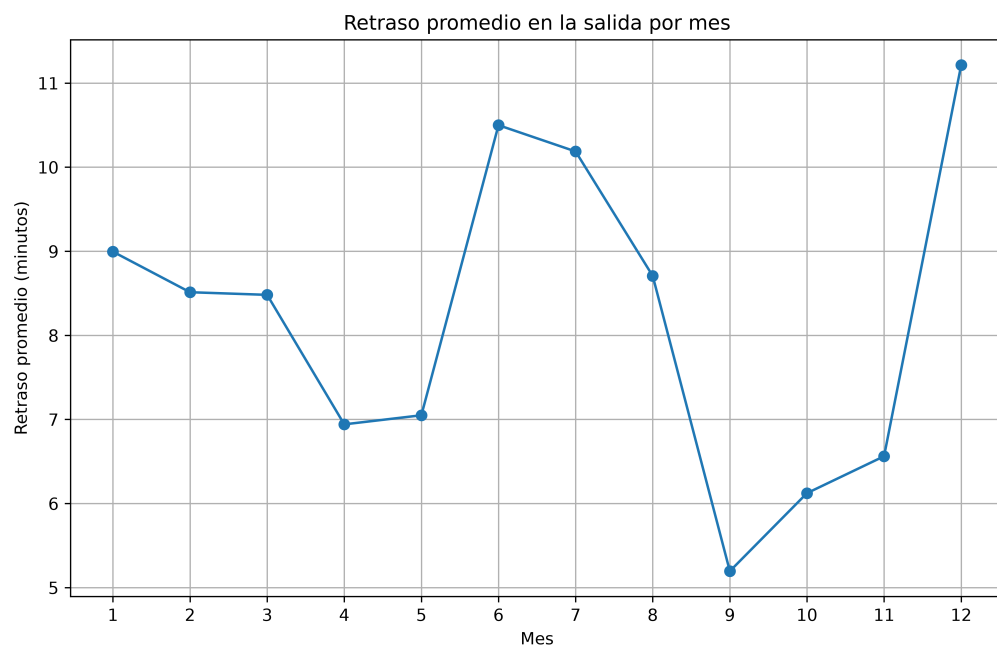
Anexo

2.1. Figuras amplias

A continuación, se vuelven a dejar todas las figuras que contienen gráficos en un mayor tamaño poder visualizarlos mejor.



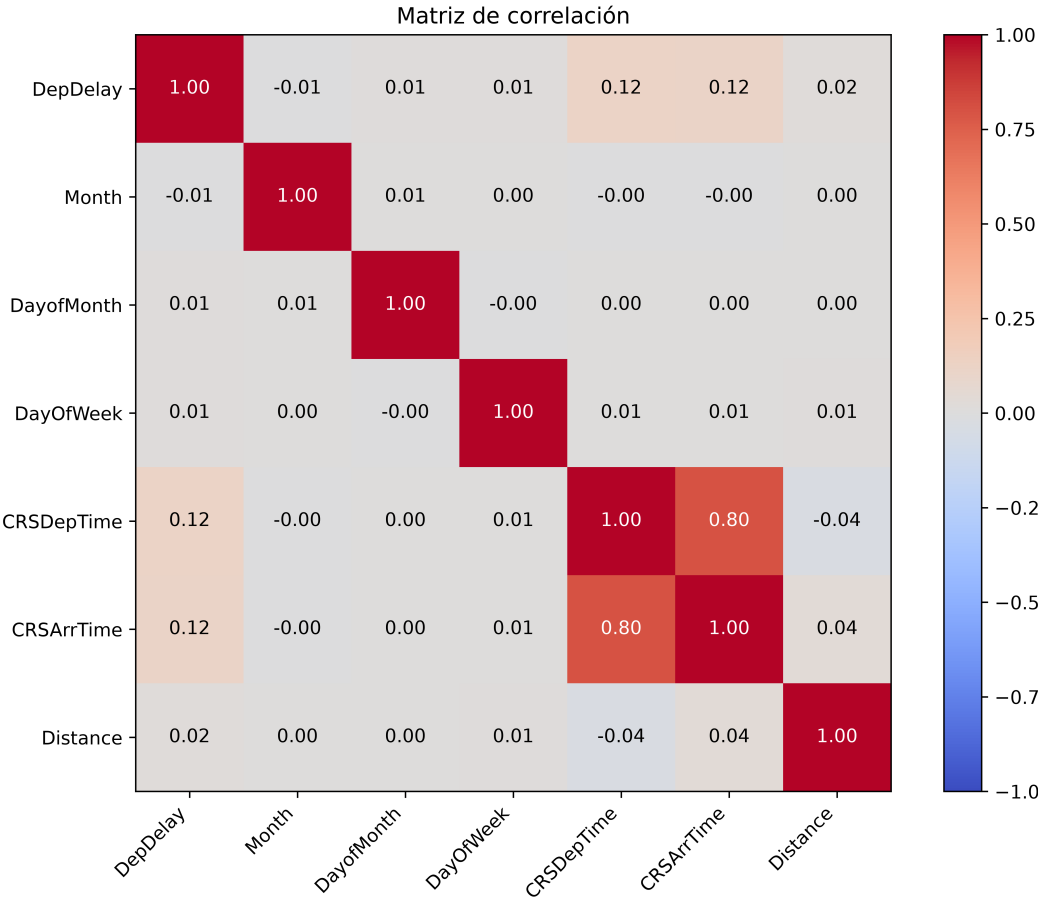




2.2. Códigos

El siguiente código sirve únicamente para la generación de los gráficos presentados, aunque no todos fueron utilizados.

Código principal.



Bibliografía

- [1] Matthew Kosinski Annie Badman. *¿Qué es Apache Parquet? / IBM* — *ibm.com*. <https://www.ibm.com/mx-es/think/topics/parquet>. [Accessed 26-08-2026].
- [2] scgupta. *Training, Validation, and Test Split for ML Models* — *ml4devs.com*. <https://www.ml4devs.com/what-is/train-validation-test-data-split/>. [Accessed 26-08-2026].